

Important SQL Queries for Data Analytics

1 Select Data

Retrieve specific columns or all rows from a table.

```
-- Select specific columns
SELECT name, age, city
FROM customers;

-- Select all columns
SELECT *
FROM customers;
```

Use SELECT to view your data.

2 Filter Data (WHERE)

Filter rows based on one or more conditions.

```
-- Customers from a specific city
SELECT *
FROM customers
WHERE city = 'Bangalore';

-- Filter with multiple conditions
SELECT *
FROM orders
WHERE order_date >= '2024-01-01'
AND status = 'Delivered';
```

Use WHERE to get only the data you need.

3 Sort Data (ORDER BY)

Sort the result set in ascending or descending order.

```
-- Sort by a column (ascending)
SELECT name, salary
FROM employees
ORDER BY salary ASC;

-- Sort by a column (descending)
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

Use ORDER BY to organize your results.

4 Limit Results (LIMIT / TOP)

Retrieve only a specific number of rows (database-specific syntax).

```
-- First 5 rows (MySQL, PostgreSQL)
SELECT *
FROM customers
LIMIT 5;

-- Top 5 rows (SQL Server)
SELECT TOP 5 *
FROM customers;
```

Use LIMIT (MySQL/PostgreSQL) or TOP (SQL Server) to view a few rows quickly.

5 Aggregate Functions

Perform calculations and summary statistics on data.

```
-- Count total customers
SELECT COUNT(*) AS total_customers
FROM customers;

-- Find average, sum, min, max
SELECT AVG(salary) AS avg_salary,
SUM(salary) AS total_salary,
MIN(salary) AS min_salary,
MAX(salary) AS max_salary
FROM employees;
```

Use aggregate functions to get summary statistics.

Note:
COUNT(*) counts all rows, while COUNT(column_name) ignores NULL values.

6 Group Data (GROUP BY)

Group data and use aggregate functions.

```
-- Total sales by city
SELECT city, SUM(sales) AS total_sales
FROM orders
GROUP BY city;

-- Average salary by department
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Use GROUP BY to analyze data in groups.

7 Filter Groups (HAVING)

Filter grouped results based on a condition.

```
-- Cities with total sales greater than 10000
SELECT city, SUM(sales) AS total_sales
FROM orders
GROUP BY city
HAVING SUM(sales) > 10000;
```

Use HAVING to filter grouped results.

8 Join Tables

Combine data from multiple tables.

```
-- Inner join: customers with their orders
SELECT c.customer_id, c.name, o.order_id, o.order_date
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.customer_id;
```

Use JOIN to combine related data from multiple tables.

9 Subqueries

Find customers with more orders than the average customer.

```
-- Customers with more orders than average
SELECT customer_id, COUNT(*) AS total_orders
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > (
    SELECT AVG(order_count) FROM (
        SELECT COUNT(*) AS order_count
        FROM orders
        GROUP BY customer_id
    ) AS temp
);
```

Subqueries help you solve complex problems.

10 Common Table Expression (CTE)

Use CTEs to make complex queries easier to read and maintain.

```
-- Find top customers by total sales using CTE
WITH customer_sales AS (
    SELECT customer_id, SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)
SELECT customer_id, total_sales
FROM customer_sales
ORDER BY total_sales DESC
LIMIT 5; -- MySQL/PostgreSQL
```

CTEs make your queries cleaner and easier to understand.



Quick Tips for SQL in Data Analytics

- ☐ Always use meaningful aliases.
- ☐ Filter data early to improve performance.
- ☐ Use aggregate functions for insights.
- ☐ Practice with real datasets.
- ☐ Keep your queries clean and readable.
- ☐ Explore different join types (INNER, LEFT, etc.).

12 DATABASE CONCEPTS

EVERY DATA PROFESSIONAL SHOULD KNOW

Better Data
Better Decisions

1 Tables

id	name	email
1	Alice	alice@mail.com
2	Bob	bob@mail.com
3	Carol	carol@mail.com

Store data in rows and columns.

2 Primary Key

id	name	email
1	Alice	alice@mail.com
2	Bob	bob@mail.com
3	Carol	carol@mail.com

Uniquely identifies each row
(and ensures unique values).

3 Foreign Key

Users		Orders		
id	name	id	user_id	total
1	Alice	101	1	99.0
2	Bob	102	2	49.0
3	Carol	103	1	75.0

Links one table to another.

4 Joins

Customers		Orders		
id	name	id	customer_id	total
1	Alice	101	1	99.0
2	Bob	102	2	49.0
3	Carol	103	1	75.0

Combine data from multiple tables.

5 Indexes

customer_id	Row location (pointer)
1	→ Row 5
2	→ Row 12
3	→ Row 18
4	→ Row 27
5	→ Row 31

Speed up data retrieval.

6 Normalization

Unnormalized			Products		
id	product	category	id	product	category_id
1	Laptop	Electronics	1	Laptop	1
2	Chair	Furniture	2	Chair	2
3	Mouse	Electronics	3	Mouse	1

Categories	
id	category
1	Electronics
2	Furniture

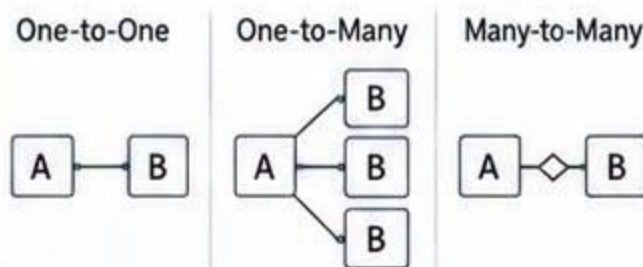
Reduces data redundancy.

7 SQL Queries

```
SELECT customer_id, SUM(total) AS total_spent
FROM orders
GROUP BY customer_id
ORDER BY total_spent DESC;
```

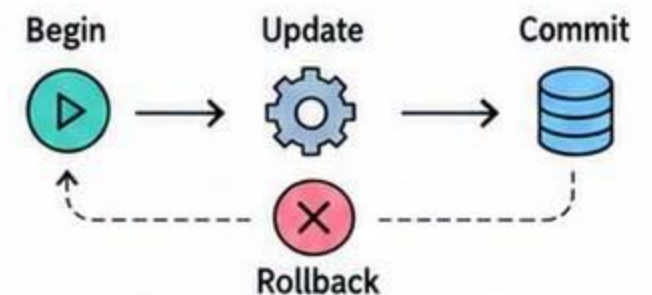
Use SQL to explore and analyze data.

8 Relationships



Define how tables are related.

9 Transactions



Ensure data changes are safe and reliable.

10 ACID Properties



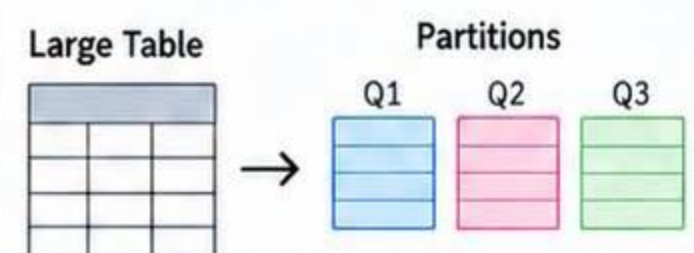
Keep transactions reliable.

11 Window Functions

id	name	score	RANK() OVER (ORDER BY score DESC)	SUM(score) OVER (ORDER BY score DESC)
1	Alice	90	1	90
2	Bob	80	2	170
3	Carol	70	3	240
4	Dave	60	4	300

Perform calculations across related rows.

12 Partitioning



Split large tables into smaller parts
for better performance.

Data Engineering Explained

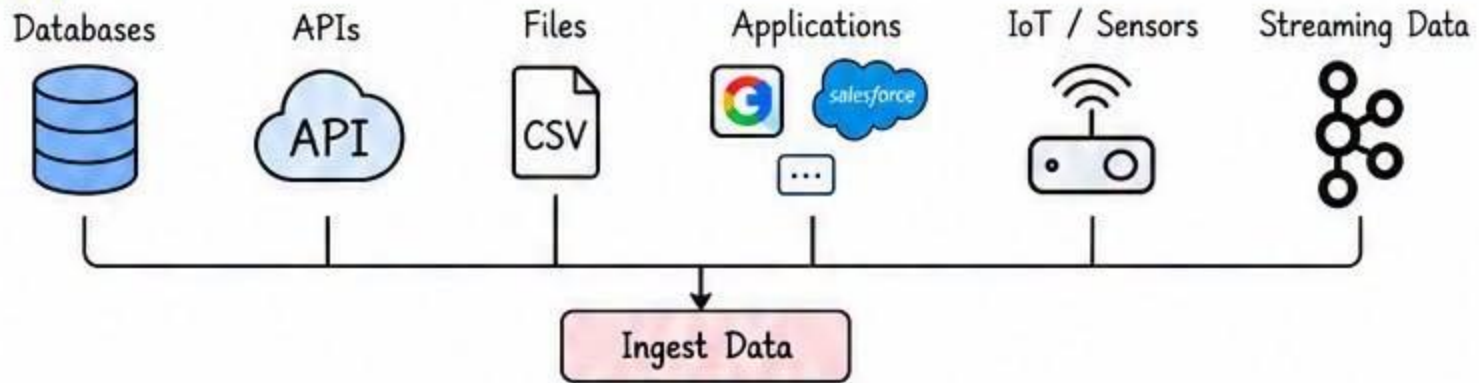
From raw data to reliable data for real-world use.

□ = Data
→ = Flow

Sources (Ingestion)

Collect data from different sources.

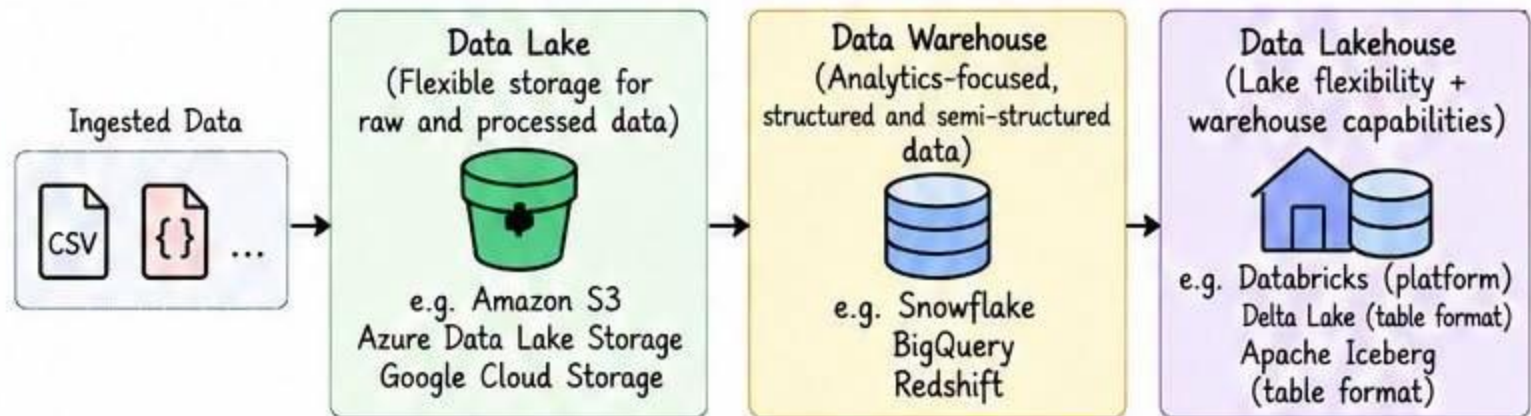
- Databases
- Applications (APIs)
- Logs, files, clickstreams
- Sensors / IoT devices
- Third-party data



2. Data Storage

Store data reliably and cost-effectively for future use.

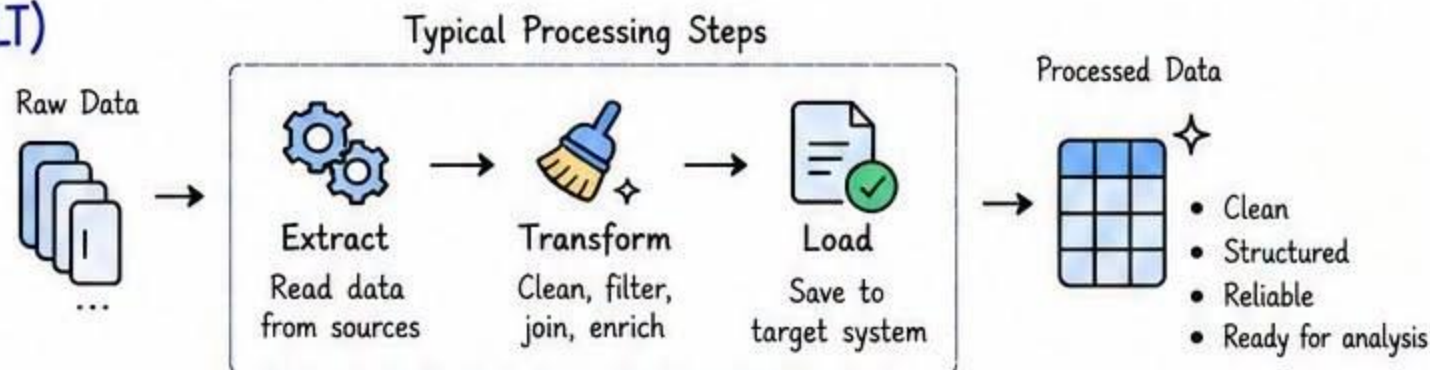
- Data Lake (raw and processed data)
- Data Warehouse (structured data)
- Data Lakehouse (both)



3. Data Processing (ETL / ELT)

Clean, transform and prepare data for analysis and usage.

- ETL (Extract, Transform, Load)
- ELT (Extract, Load, Transform)
- Batch and real-time processing
- Data quality checks

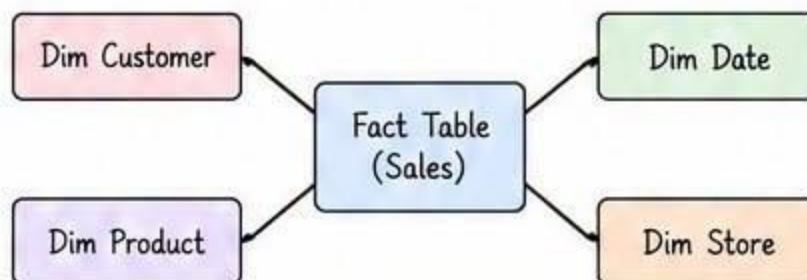


4. Data Modeling & Organization

Organize data so it's easy to understand, query and use.

- Design data models (star schema, etc.)
- Define tables and relationships
- Partitioning, clustering & optimization
- Manage metadata

Example: Star Schema



Data Catalog & Metadata



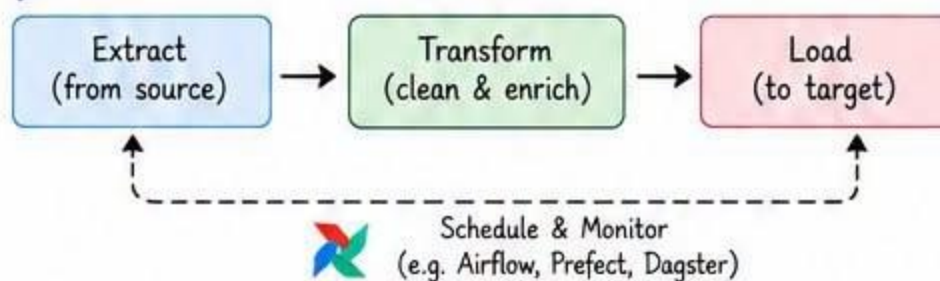
- Discover data
- Understand data
- Track lineage
- Manage access

5. Data Orchestration & Pipelines

Automate and schedule data workflows.

- Build data pipelines
- Schedule and monitor jobs
- Handle failures and retries
- Manage dependencies

Example Data Pipeline



Pipeline & Transformation Tools



Transformation Tool



6. Data Serving & Consumption

Make data available to analysts, applications, and AI/ML systems.

- SQL queries and BI tools
- APIs for data access
- Data for ML and analytics
- Self-service data for teams

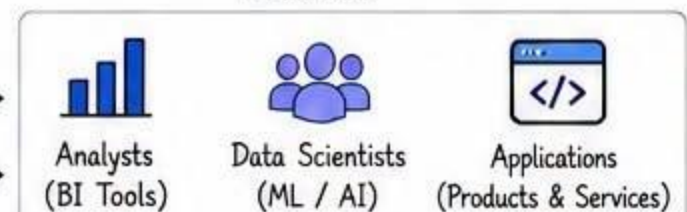
Processed Data



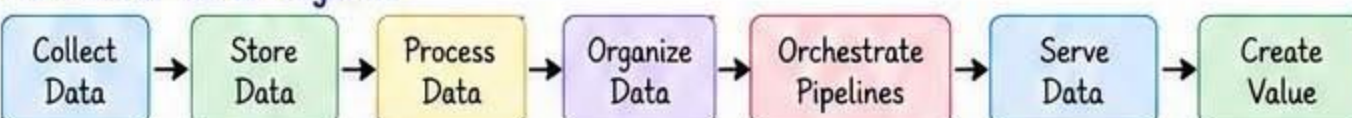
Data Access Layer

- SQL
- APIs
- Views / Semantic Layer

End Users



How It All Works Together



Key Takeaway

Data engineering builds the foundation for data-driven decisions by turning raw data into clean, reliable and accessible data.

THE FULL DATA ANALYTICS STACK EXPLAINED

